Interrogation Room: Catching Lying LLM Agents with Logic Programming

Arushi Nirmal
Department of Computer Science, San José State University, San José, CA 95192
arushi.nirmal@sjsu.edu

Abstract

LLMs give fluent, confident answers that are often logically inconsistent. *Interrogation Room* is a murder-mystery game that turns this into a test bed: each suspect is an LLM agent with private knowledge, and the culprit is told to lie. Every statement the player hears is translated into Prolog facts, and a logic engine checks for contradictions in real time, producing a proof whenever a lie is exposed. The project combines logic, functional, and object-oriented paradigms and evaluates whether symbolic verification catches deception more reliably than an LLM judge.

Keywords: Neuro-Symbolic AI, Logic Programming, Prolog, Multi-Agent LLMs, Hallucination Detection

1 Body of Work

1.1 Problem

LLMs hallucinate and contradict themselves [1], and an LLM judge inherits the same flaws [4]. Pairing LLMs with symbolic solvers improves faithful reasoning [2], but mostly on static benchmarks; a live, adversarial setting is a more realistic test of logic as a guardrail.

1.2 Proposed Approach

- **Suspect agents:** each suspect is an object with private state (alibi, secrets, relationships) and a persona prompt, following the generative-agents pattern [3]. The culprit is told to lie convincingly.
- **Language-to-logic translator:** an LLM with structured (JSON) output converts each answer into Prolog facts such as `location(butler, kitchen, 21).`
- **Truth engine:** SWI-Prolog [5] alone decides what is true; rules over time, location, evidence, and testimony detect contradictions and return a proof trace explaining the exposed lie.
- **Mystery generator + UI:** an LLM writes new scenarios that Prolog verifies have exactly one solution; a Streamlit app shows the chat and a live evidence board.

1.3 Alignment with Programming Paradigms

Logic (Prolog facts, rules, unification, backtracking as the reasoning core), **functional** (pure functions that parse, validate, and transform statements into facts), and **object-oriented** (agents with encapsulated private state). The report compares how each fits its role.

1.4 Evaluation and Milestones

Across 20+ generated mysteries, lie-detection accuracy and false-alarm rate will be compared for (1) the Prolog checker, (2) an LLM judge [4], and (3) a pure-LLM detective. Milestones: Prolog mystery (wk 1–2), agent + translator (wk 3–4), live multi-agent checking (wk 5–6), generator + evaluation (wk 7–8), UI + report (wk 9).

References

[1] Z. Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023.
[2] L. Pan et al. Logic-LM: Empowering LLMs with symbolic solvers. *Findings of EMNLP*, 2023.
[3] J. S. Park et al. Generative agents: Interactive simulacra of human behavior. *ACM UIST*, 2023.
[4] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS D&B*, 2023.
[5] J. Wielemaker et al. SWI-Prolog. *Theory and Practice of Logic Programming*, 12(1–2), 2012.

Preprint.